

Applications and Offer Status Analysis

Given two datasets, 'applications' and 'offers', which indicates the applications status and offers statuses respectively, let us dive into how they relate with each other, and how we can discover insights on how loans can be approved more efficiently.

```
print(applications.head())
```

	salesName	applic_status_name	age	appliedAmount	civilStatus	
0	NaN	Closed App	32	42000.0	single	
1	NaN	Closed App	29	10000.0	married	
2	Sales rep C	Expired App	51	60000.0	married	
3	Sales rep C	Expired App	52	10000.0	married	
4	Sales rep C	Closed App	33	2000.0	married	

	loanPurpose	monthlyIncome	nationality	residency	workStatus	
0	debt-consolidation	3200.0	SG	s	employed-salary	
1	debt-consolidation	3000.0	SG	s	employed-salary	
2	business-expansion	6200.0	SG	s	self-employed	
3	debt-consolidation	3120.0	SG	s	employed-salary	
4	paying-bills	3400.0	SG	s	employed-salary	

	jobIndustry	applicationTime	appId	leadSource	
0	government	2022-03-17 13:28:48.513	App 0001	Google	
1	government	2022-03-17 13:28:48.512	App 0002	Google	
2	retail	2022-01-05 14:37:25.759	App 0003	e-mail	
3	government	2022-01-07 23:13:56.893	App 0004	google	
4	it-telco	2022-01-05 19:28:59.264	App 0005	google	

```
print(offers.head())
```

	pendingOfferTime	offerGivenTime	offerChosenTime	
0	NaN	NaN	NaN	
1	2022-04-01 02:08:26.178	2022-04-01 02:25:48.247	NaN	
2	NaN	NaN	NaN	
3	NaN	NaN	NaN	
4	2022-04-01 02:08:26.179	2022-04-01 15:29:56.263	NaN	

	disburseTime	offer_status_name	lenderName	appId	offerId
0	NaN	Queued	Lender 01	App 3692	8030317a
1	NaN	Bank Approved	IPA Lender 02	App 3692	80303179
2	NaN	Queued	Lender 03	App 3692	80303178
3	NaN	Queued	Lender 04	App 3692	80303177
4	NaN	Bank Approved	IPA Lender 05	App 3692	80303176

```
print(applications.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3692 entries, 0 to 3691
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   salesName              3630 non-null  object
1   applic_status_name     3692 non-null  object
2   age                   3692 non-null  int64
3   appliedAmount          3692 non-null  float64
4   civilStatus            3692 non-null  object
5   loanPurpose            3692 non-null  object
6   monthlyIncome          3692 non-null  float64
7   nationality             3692 non-null  object
8   residency              3692 non-null  object
9   workStatus             3692 non-null  object
10  jobIndustry            3685 non-null  object
11  applicationTime         3692 non-null  object
12  appId                  3692 non-null  object
13  leadSource              3692 non-null  object
dtypes: float64(2), int64(1), object(11)
```

```
print(offers.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 131665 entries, 0 to 131664
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   pendingOfferTime       61191 non-null  object
1   offerGivenTime         25601 non-null  object
2   offerChosenTime        2023 non-null   object
3   disburseTime           610 non-null    object
4   offer_status_name      131665 non-null object
5   lenderName             131665 non-null object
6   appId                  131665 non-null object
7   offerId                 131665 non-null object
dtypes: object(8)
```

From the initial data inspection, we can see that the 'applications' dataset holds categorical information such as civil status, loan purposes and application status name, as well as numerical columns appliedAmount, monthlyIncome and age. The 'offers' dataset holds all categorical columns, namely offer given time, offer status and application ID. Let's proceed to clean the data.

Data Cleaning

```
print(applications.isnull().sum())
✓ 0.0s
```

salesName	62
applic_status_name	0
age	0
appliedAmount	0
civilStatus	0
loanPurpose	0
monthlyIncome	0
nationality	0
residency	0
workStatus	0
jobIndustry	7
applicationTime	0
appId	0
leadSource	0
dtype:	int64

```
applications[applications['jobIndustry'].isna()]
✓ 0.0s
```

	salesName	applic_status_name	age	appliedAmount	civilStatus	loanPurpose	monthlyIncome
7	Sales rep F	Closed App	40	50000.0	married	debt-consolidation-ff	4500.0
1231	Sales rep F	Closed App	75	14000.0	married	renovation	2000.0
1330	Sales rep I	Bank Declined App	49	20000.0	married	paying-bills	3500.0
2411	Sales rep F	Bank Declined App	22	1500.0	single	paying-bills	2500.0
3442	Sales rep I	Bank Declined App	39	30000.0	single	debt-consolidation	4000.0
3508	Sales rep F	Bank Declined App	35	140000.0	married	paying-bills	2000.0
3666	Sales rep F	Closed App	39	20000.0	single	other	2000.0

After inspecting the data, we can see that there are a few null values at the job industry column, and this is would be considered normal because the application status names are either closed or declined by the bank, indicating that all applicants with no jobs are declined of their applications. We also proceed to convert the applicationTime to DateTime format for further usage.

```
applications['applicationTime'] = pd.to_datetime(applications['applicationTime'])
# applications['application_month'] = applications['applicationTime'].dt.month
# applications['days_since_application'] = (pd.to_datetime('now') - applications['applicationTime']).dt.days
applications.head()
✓ 0.0s
```

	salesName	applic_status_name	age	appliedAmount	civilStatus	loanPurpose	monthlyIncome
0	NaN	Closed App	32	42000.0	single	debt-consolidation	
1	NaN	Closed App	29	10000.0	married	debt-consolidation	
2	Sales rep C	Expired App	51	60000.0	married	business-expansion	
3	Sales rep C	Expired App	52	10000.0	married	debt-consolidation	
4	Sales rep C	Closed App	33	2000.0	married	paying-bills	

```
# Check unique values for categorical columns in offers
print(offers['offer_status_name'].unique())
print(offers['lenderName'].unique())
✓ 0.0s
```

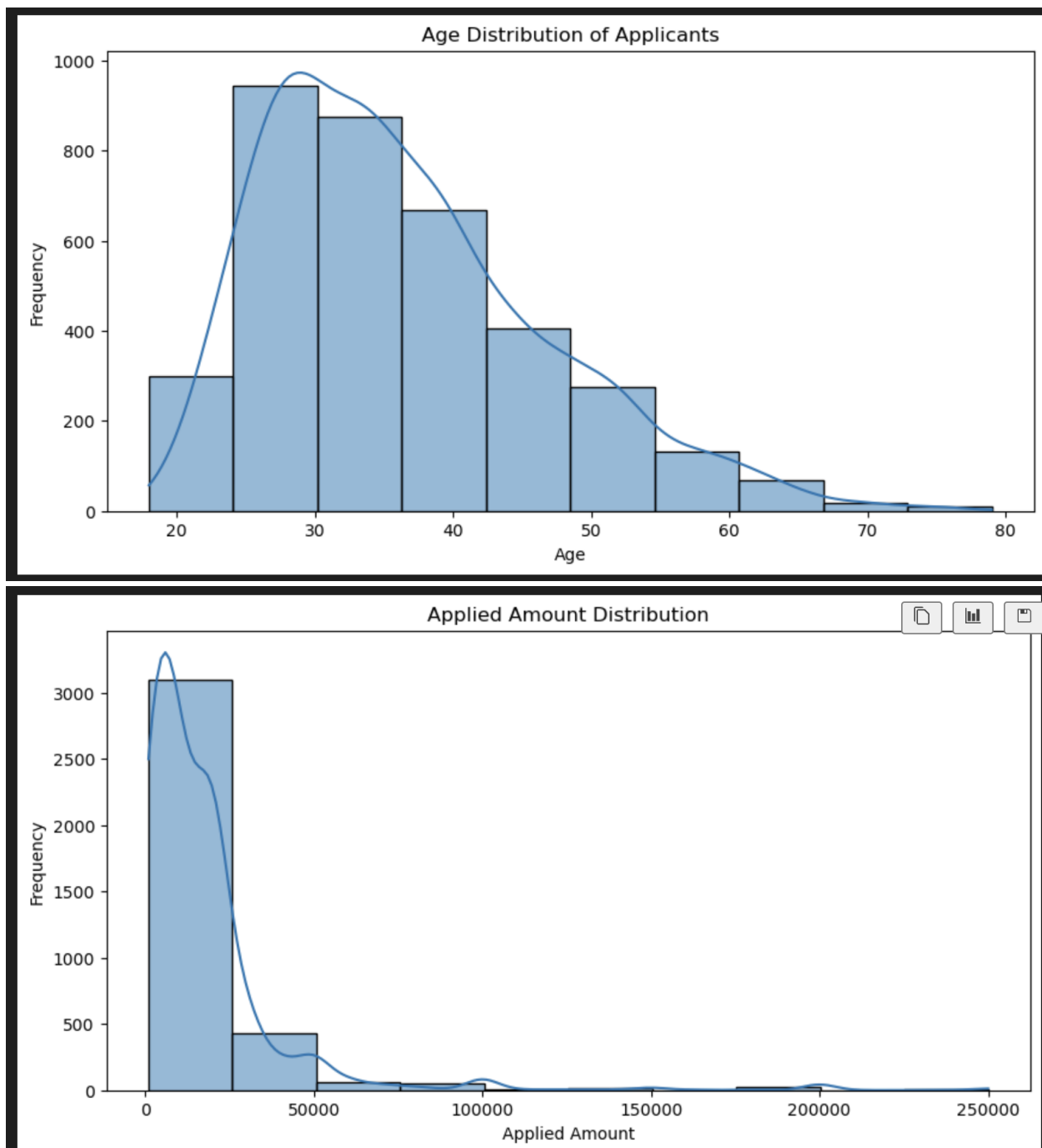
['Queued' 'Bank Approved IPA' 'Bank Declined IPA' 'Pending IPA'
'Loan Disbursed IPA/LG' 'PR Fail IPA/LG' 'Customer Reject IPA/LG'
'Loan Rejected - Post' 'Loan Rejected - Pre'
'Chosen - Lender Review' 'Acknowledged'
'Chosen - Manual Review']

['Lender 01' 'Lender 02' 'Lender 03' 'Lender 04' 'Lender 05' 'Lender 06'
'Lender 07' 'Lender 08' 'Lender 09' 'Lender 10' 'Lender 11' 'Lender 12'
'Lender 13' 'Lender 14' 'Lender 15' 'Lender 16' 'Lender 17' 'Lender 18'
'Lender 19' 'Lender 20' 'Lender 21' 'Lender 22' 'Lender 23' 'Lender 24'
'Lender 25' 'Lender 26' 'Lender 27' 'Lender 28' 'Lender 29' 'Lender 30'
'Lender 31' 'Lender 32' 'Lender 33' 'Lender 34' 'Lender 35' 'Lender 36']

```
offers['offer_status_name'] = offers['offer_status_name'].str.strip()
✓ 0.0s
```

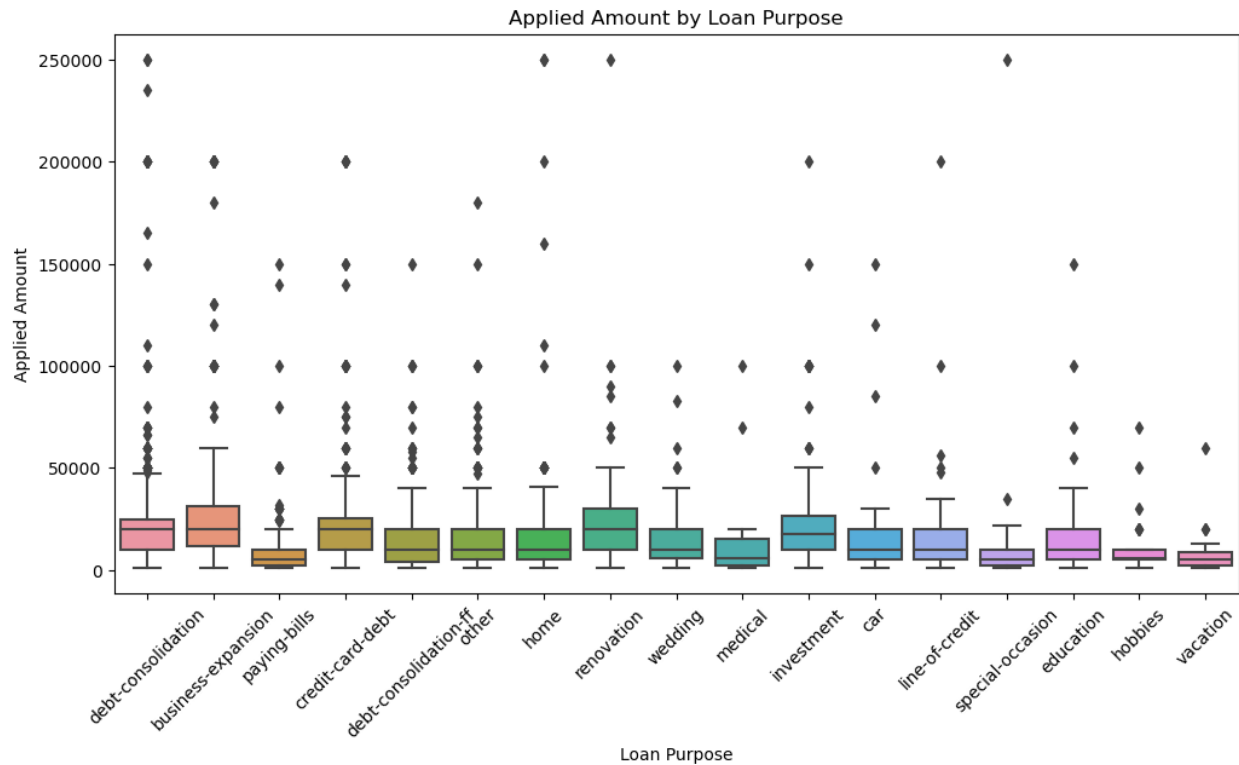
We have also observed that the offer_status_name in 'offers' have a value that contains white space, so we shall proceed to strip the leading white space for better visual purposes. It seems that the data is fit for analysis now.

Analysis



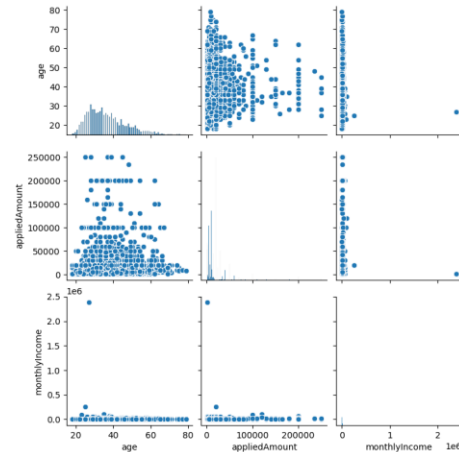
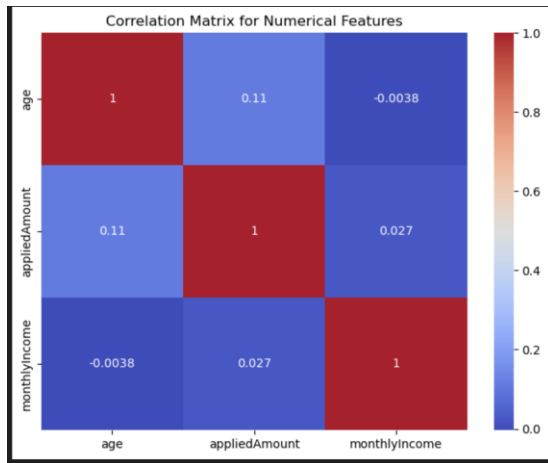
From the distribution graphs, we can confirm that there are more young applicants than older applicants, as most of the applicants fall in the range of 20-40 years old. The applied amount is skewed to the right skewed also, indicating that the mode and median is greater than the mean (the highest applied amount is around 10000-20000).

We would also like to see how the applied amount affects the loan purposes.

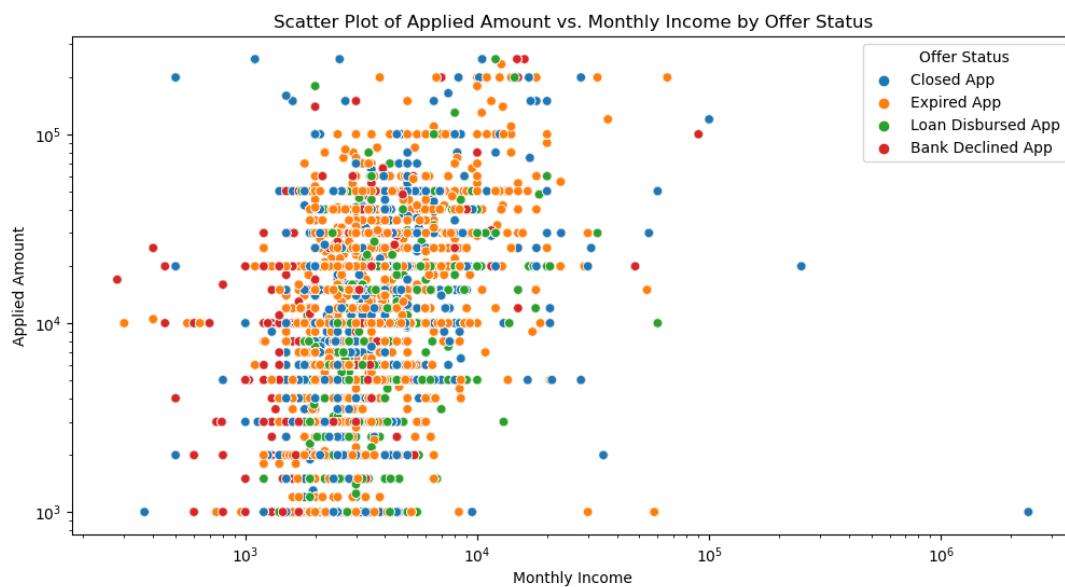


From this boxplot, we can identify that the median or central tendency of the applied amount of each loan purposes is around the same, falling below 50000. We can also see that the business-expansion, renovation and investment have the largest spread among the categories, so to say that these purposes tend to have a larger applied amount of loan. Debt consolidation, credit card debt, and home have the most outliers as well, which is understandable as debts may incur unusual borrowing patterns.

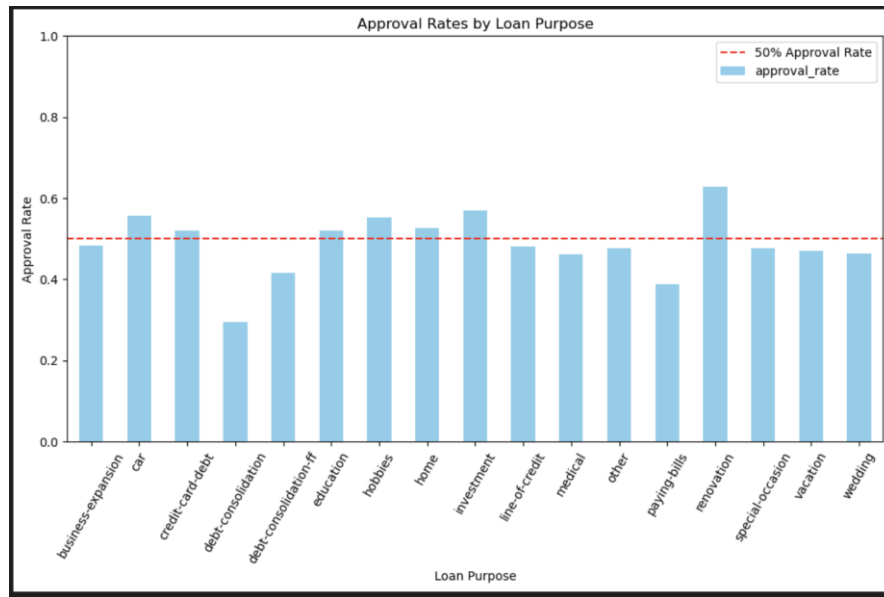
Let us look at how the numerical columns relate to each other.



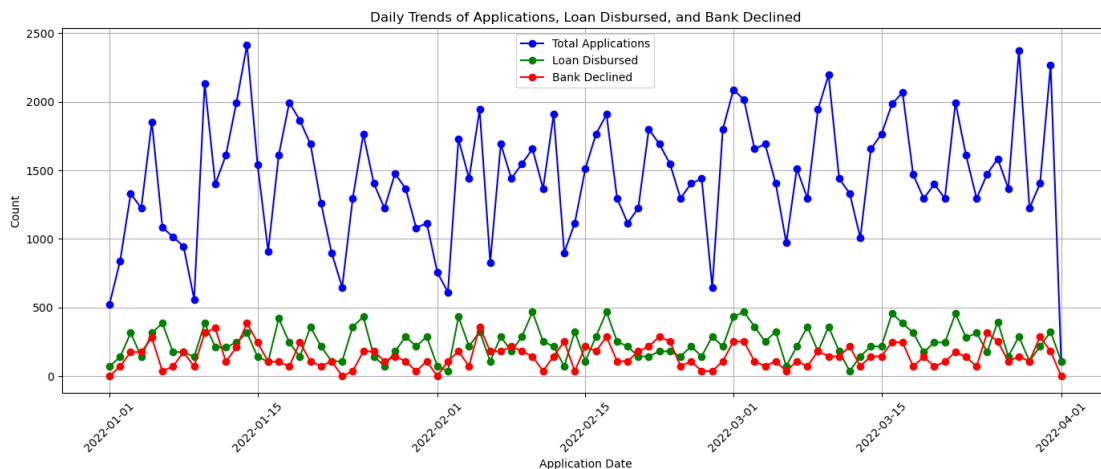
From both the correlation matrix and pairplot, there is tiny to none correlation between the columns. The highest relationship we can observe would be the appliedAmount and age, a good explanation would be that the applied amount increases as age increases. However, from the pairplot, we can hardly see any correlation indicating this. Let us dive deeper into more analysis to find out.



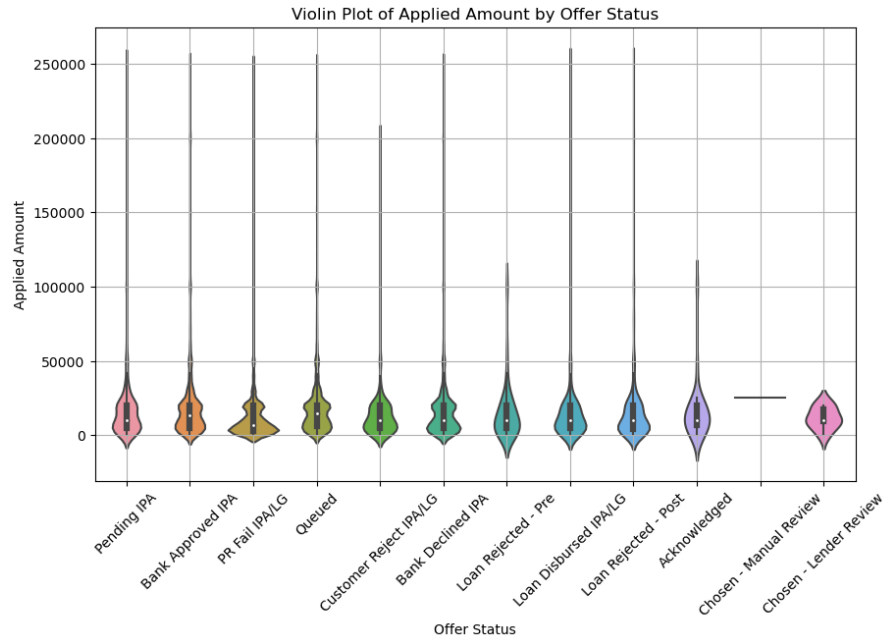
We added another indicator offer status into the scatter plot, hoping it would show us more insights on the whether the applied amount and monthly income have a correlation, together with if they are declined/disbursed. The dispersion is almost horizontal and too large, indicating that these two variables show little to no correlation. We can also observe more shades of green when the applied amount is lower, indicating a lower applied amount will have a higher chance of disbursed loans.



This visualization shows how different loan purposes affect the likelihood of application approval. A high approval rate for certain loan purposes may indicate that these types of loans are less risky or more favorable to lenders, in this case renovation, investments, cars, credit card debt, home and education seems the most favorable for lenders to offer loans. On the other hand, debt consolidation and bills are the least favorable, indicating that lenders should have a good financial profile for loans to be approved.



This line chart provides a clear view of how the number of applications and accepted offers changes over time. We can observe that the number of applicants is the highest at January, which then declined stating from February. From the points observed, it seems that when the total number of applicants increases, the number of banks declined loans decreases, indicating that there are more loans approved than declined in the sample.



From the heatmap and violin graph, we want to find out how the offer status relates to loan purposes and applied amount, respectively. From the violin plot, we observe that the width of the violin for 'Bank Declined' and 'Loan Disbursed' is non identical, and that the range for 'Bank Declined' is a smaller width than 'Loan Disbursed', concluding our findings to be true :- a lower applied amount will lead to a higher chance of loan disbursing. 'Acknowledged', 'Customer Reject' and 'Loan Rejected – Pre' have a smaller density and range, indicating that these loans of smaller amounts are most likely not that important to customers such that they are in the pending or already rejected by applicants before screening processes.

The heatmap shows us debts and paying bills for loan purposes have the largest correlation with the bank declining the application. However, we can observe that the credit card debt purpose has

an almost similar correlation for bank approved and bank declined, indicating that there are other factors, in this case applied amount and income which determines the offer status.

Data Modelling

Next, let us try to perform data modelling to predict the offer status of our incoming new dataset. We will be using logistic regression and also decision tree, since there are multiple offer statuses that needs to be modelled. With Logistic Regression, we have managed to achieve an accuracy of 0.46, which is not too ideal. From the confusion matrix and report, we can see that the model has made quite a few mistakes in predicting the dataset. Our model successfully predicted variables with 'PR Fail IPA/LG' and 'Queued' the most, however we are more interested in variables such as Bank Approved and Bank Declined. Hence, we shall try with decision tree next.

```
accuracy_logistic = accuracy_score(y_test, y_pred_logistic)
confusion_logistic = confusion_matrix(y_test, y_pred_logistic)
report_logistic = classification_report(y_test, y_pred_logistic)

print("Logistic Regression Accuracy:", accuracy_logistic)
✓ 0.7s
```

[/opt/anaconda3/lib/python3.9/site-packages/sklearn/metrics/_classification.py:136: UserWarning:](#)
_warn_prf(average, modifier, msg_start, len(result))
Logistic Regression Accuracy: 0.46360080507348195
[/opt/anaconda3/lib/python3.9/site-packages/sklearn/metrics/_classification.py:136: UserWarning:](#)
_warn_prf(average, modifier, msg_start, len(result))
[/opt/anaconda3/lib/python3.9/site-packages/sklearn/metrics/_classification.py:136: UserWarning:](#)
_warn_prf(average, modifier, msg_start, len(result))

Classification Report:				
	precision	recall	f1-score	support
Acknowledged	0.00	0.00	0.00	13
Bank Approved IPA	0.25	0.00	0.01	4703
Bank Declined IPA	0.40	0.29	0.34	6205
Customer Reject IPA/LG	0.00	0.00	0.00	116
Loan Disbursed IPA/LG	0.00	0.00	0.00	135
Loan Rejected - Post	0.00	0.00	0.00	146
Loan Rejected - Pre	0.00	0.00	0.00	7
PR Fail IPA/LG	0.66	0.57	0.62	5152
Pending IPA	0.33	0.00	0.01	902
Queued	0.43	0.83	0.57	8954
accuracy			0.46	26333
macro avg	0.21	0.17	0.15	26333
weighted avg	0.43	0.46	0.39	26333


```

accuracy_tree = accuracy_score(y_test, y_pred_tree)
confusion_tree = confusion_matrix(y_test, y_pred_tree)
report_tree = classification_report(y_test, y_pred_tree)

print("Decision Tree Classifier Accuracy:", accuracy_tree)
✓ 0.8s

/opt/anaconda3/lib/python3.9/site-packages/sklearn/metrics/cla
_warn_prf(average, modifier, msg_start, len(result))
Decision Tree Classifier Accuracy: 0.5451714578665553
/opt/anaconda3/lib/python3.9/site-packages/sklearn/metrics/cla
_warn_prf(average, modifier, msg_start, len(result))
/opt/anaconda3/lib/python3.9/site-packages/sklearn/metrics/cla
_warn_prf(average, modifier, msg_start, len(result))

```

With a decision tree, we managed to perform better, with 0.54 accuracy. We can increase this by performing grid search and feature selection for further improvement. From the feature importances, we see that residency with spass-wp and also monthly income have the largest effect for loan prediction.

```

Feature Importances:

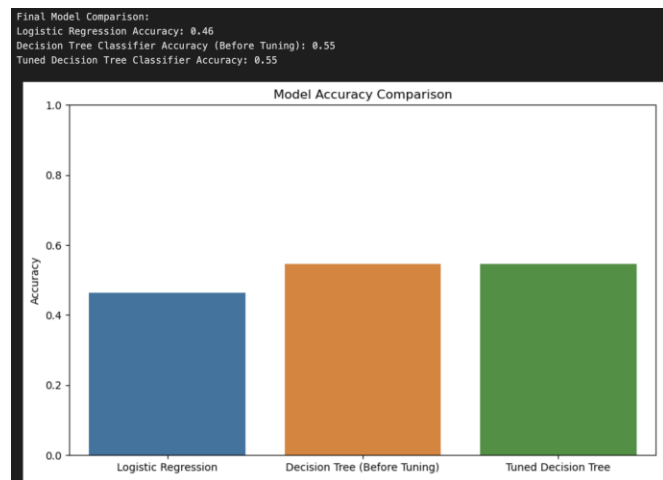
```

	Feature	Importance
57	residency_spass-wp	0.278401
2	monthlyIncome	0.276121
0	age	0.087921
1	appliedAmount	0.049917
11	loanPurpose_debt-consolidation	0.030449
..	...	...
32	nationality_HK	0.000000
25	nationality_AU	0.000000
78	leadSource_best_in_singapore	0.000000
43	nationality_NO	0.000000
26	nationality_AX	0.000000

```

[111 rows x 2 columns]

```



Comparing the three models used, we can say that the decision tree has the best accuracy, and by using grid search and also feature selection, we only managed to boost the accuracy by little (0.1), hence more methods can be considered such as DNN, CNN or Random Forests.

In conclusion, although from our matrixes we can hardly find any correlation between all these variables, by diving deeper into the plots, we can observe there are indeed certain variables such as applied amounts, age, job statuses and incomes that will affect the application status of the loans. Therefore, further analysis and methodologies can be explored to obtain better insights.